

Meta-Diffusion: Implementation Specification and Results to Date

Abstract

This report documents the network architecture, dataset organisation, the complete flow of data through both the meta-training and the deployment phase, the hyperparameter settings, and the term-by-term definition of the loss functions of the current Meta-Diffusion implementation. Every figure quoted here was exported from the running code rather than recalled. One point should be made at once: **every experimental result to date derives from two-dimensional Gaussian mixtures, and no substantive training has yet been carried out on images**; Section 2 sets this out.

Contents

1	Network architecture	2
2	Dataset structure	4
2.1	What data is presently in use	4
2.2	Composition of the Gaussian-mixture data	4
2.3	The CIFAR-100 partition scheme	6
2.4	Three departures from the specified protocol	7
3	Data flow in both phases	7
3.1	Training	8
3.2	Deployment	8
4	Hyperparameters and module specifications	9
5	Loss functions	9
5.1	Meta-training	10
5.2	Meta-testing	11
6	Results to date	11
6.1	Statistical significance	12
6.2	Quantitative test of the low-dimensionality hypothesis	12
6.3	The effect of task-family relatedness	13
6.4	Structural diagnostics	13
6.5	Three methodological cautions	13
	update 09/09/2026	15
7	Framework structure	15
8	Dataset construction	15
8.1	Within one fine class	15
8.2	How the target domain is made	15

9	Data flow	17
10	Loss functions	18
10.1	Notation	18
10.2	The three coordinates	18
10.3	The predictor each loss is built on	18
10.4	Term 1 — source denoising loss	19
10.5	Term 2 — target denoising loss, encoder coordinate	19
10.6	Term 3 — transport denoising loss	19
10.7	Term 4 — coordinate regulariser	19
10.8	The total	19
10.9	What is actually computed each step	19
10.10	The deployment objective	20
11	Experiments run	20
11.1	What the numbers in these tables are	20
11.2	E1 — analytic validation on two-dimensional Gaussian mixtures	21
11.3	E2 — CIFAR-100, sibling fine classes	21
11.4	E3 — CIFAR-100, one fixed transformation	22
11.5	E4 — CIFAR-100, three transformations	22
11.6	E5 — coordinate dimension sweep	22
11.7	E6 — backbone capacity sweep	22
11.8	E7 — convergence of the small backbones	23
11.9	E8 — target-support and source-evidence sweeps	23
11.10	E9 — mean-coordinate substitution	24
11.11	E10 — loss sensitivity to coordinate error	24
11.12	E11 — coordinate accuracy against an abundant reference	24
12	Reproduction procedure	25

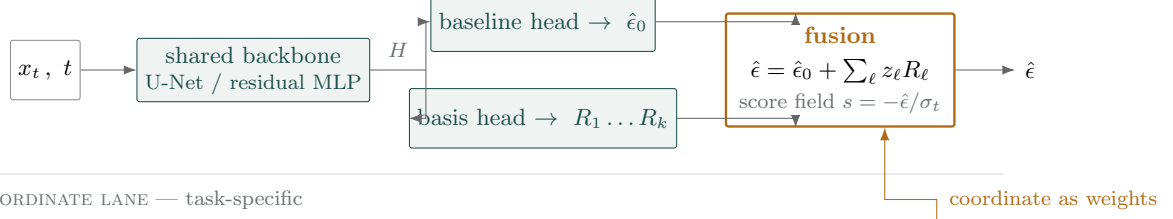
1 Network architecture

The framework comprises four modules, organised according to a single principle of division: **everything high-dimensional is shared across all tasks, and everything task-specific is compressed into one low-dimensional vector**. The shared portion is frozen entirely at deployment; the task-specific portion is the only object that deployment optimises.

Computationally this division reduces to an addition. A shared backbone network produces one feature tensor; two prediction heads read from that tensor a baseline noise prediction and a set of noise-perturbation directions respectively. The task coordinate then serves as a vector of weights, linearly combining the latter and adding the result to the former. The expressive power of the entire method therefore rests on a single question — whether those learned directions can span the differences that actually separate one task from another.

The asymmetry between the two lanes is the point of the whole design. The backbone lane moves high-dimensional tensors and carries tens of millions of parameters; the coordinate lane terminates in a little over a dozen scalars. The former holds what all tasks have in common; the latter holds this one task’s departure from that commonality.

BACKBONE LANE — task-independent, frozen at deployment



COORDINATE LANE — task-specific



Figure 1: Architecture. Green modules are shared across all tasks and frozen at deployment; amber denotes the task-specific coordinate, the only object optimised at deployment (k scalars). H is the shared feature tensor read by both heads. The transport network sits at the end of the coordinate lane and **receives no target-domain sample whatsoever** — it must predict the target coordinate having never observed target data, which is precisely the proposition under examination.

Module	Function	Governing constraint
Shared backbone (frozen)	Maps a noised sample and timestep to a shared feature tensor	Contains no noise-prediction head of its own. A single interface separates it from everything above, so substituting a different backbone requires no change elsewhere
Baseline head (frozen)	Reads the baseline noise prediction from the shared features	Shares one backbone pass with the basis head: one backbone network rather than k independent denoisers
Basis head (frozen)	Reads k noise-perturbation directions from the same features	Output channel count equals k times the image channel count; reshaped into k tensors of image shape
Set encoder (frozen)	Infers a coordinate from a set of samples	Permutation invariant. Source and target share one encoder ; there is no separate branch
Transport network (frozen)	Predicts the target coordinate from the source coordinate	Residual in form; operates purely within the k -dimensional coordinate space. Its input contains no target-domain sample
Task coordinate (optimised)	Weights the k perturbation directions	The sole degree of freedom at deployment: k scalars

Table 1: Responsibilities of the four modules.

2 Dataset structure

2.1 What data is presently in use

It should be stated at the outset that **every experimental result to date derives from two-dimensional Gaussian mixtures; no substantive training has yet been carried out on images**. The CIFAR-100 partition is complete and has been verified image by image, and the training pipeline is operational and passes every conformance check, but it has been run only for a few hundred smoke-test steps and yields no reportable result.

Data	Status	Remark
Two-dimensional Gaussian mixtures	Gaus- all results to date	The analytic validation stage. The true score field admits a closed form, so model output can be compared against ground truth directly
CIFAR-100	built, not trained	Partition complete and verified; the pipeline passes conformance checks but has not been trained to a reportable state
dSprites	not begun	Following the source document’s implementation order the image stages should begin here: its generative factors are known, so recovery of the target factors can be measured directly

Table 2: Present state of each dataset.

2.2 Composition of the Gaussian-mixture data

It must be emphasised that this data is **not random noise but samples from distributions of definite structure**. Each semantic task is a four-component Gaussian mixture whose weights, component means and covariance matrices are fixed parameters,

$$p_{y,S}(x) = \sum_{j=1}^4 \pi_{y,j} \mathcal{N}(x; \mu_{y,j}, \Sigma_{y,j}), \quad x \in \mathbb{R}^2. \quad (1)$$

The point cloud drawn from it forms four separated clusters: measured across all test tasks, the mean distance between components is roughly **eight times** the spread within a component. Isotropic Gaussian noise, by contrast, is unimodal and exhibits no cluster structure whatever.

Noise enters this method at an unrelated point: the ϵ of the forward diffusion process $x_t = \alpha_t x_0 + \sigma_t \epsilon$ is the noise, whereas x_0 is a sample from the distribution described above. This holds identically for images and for two-dimensional points; the same noising process is applied on CIFAR-100.

How a task is constructed

A semantic task consists of a source distribution together with its target counterpart, both generated by the deterministic procedure set out in Table 3. The four relations of Table 4 are transformations with fixed parameters and therefore *recur across tasks* — which is precisely what makes the relation reusable, and hence what the whole approach depends upon.

The family comprises 192 training tasks and 32 test tasks. The mixture configurations on the two sides are generated from different random streams and are therefore wholly distinct, while the four relations are shared between them. This realises the source document’s requirement exactly: **the semantic task is unseen at test time, while the relation must be reusable**.

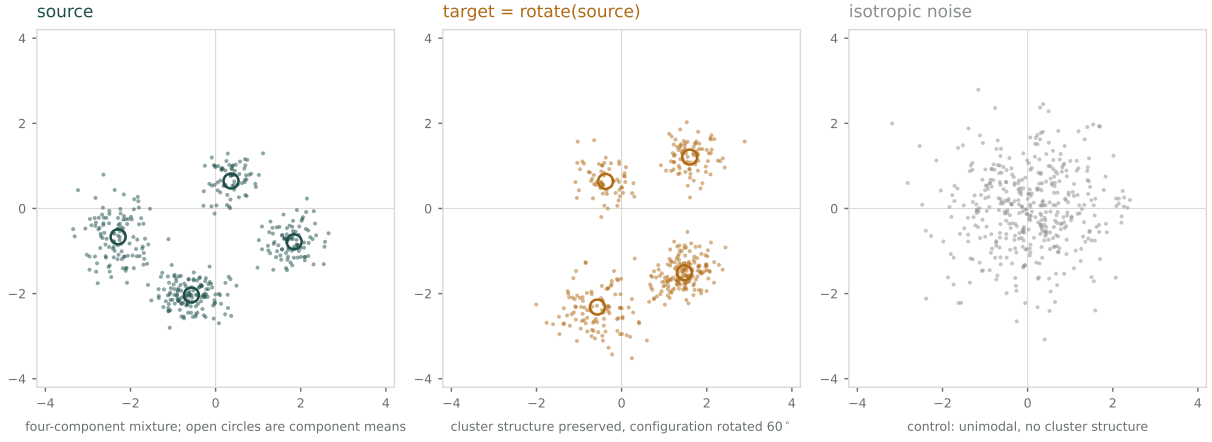


Figure 2: One instance drawn from the task family (test task 196, relation: rotation). The points in the left and centre panels are sampled from the actual distributions rather than drawn for illustration; the four open circles mark the component means. The centre panel is the left rotated bodily through sixty degrees while the relative arrangement of the clusters is preserved — and it is precisely this pattern, structure invariant under a global transformation, that the transport network must learn. The right panel shows isotropic Gaussian noise at the same sample size for contrast.

Element	How it is drawn	Role
Component means	Four components, each drawn uniformly on $[-2.5, 2.5]^2$	Fixes the positions of the four clusters
Covariances	Diagonal of a lower-triangular factor uniform on $[0.15, 0.5]$, off-diagonal uniform on ± 0.075 , then $\Sigma = LL^\top$	Positive definite by construction, while permitting each cluster its own shape and orientation
Mixture weights	Four standard normal draws scaled by 0.5, passed through a softmax	The clusters carry unequal mass, typically between 0.15 and 0.35
Source-to-target transform	One of four relations, drawn with equal probability	See Table 4

Table 3: Construction of one semantic task.

Relation	Action	Effect on the distribution
rotate	Component means and covariances rotated 60° about the origin	The configuration turns as a whole; the relative arrangement of clusters is preserved
translate	All component means shifted by $(+1.5, -0.8)$	The configuration is displaced; shapes and weights unchanged
scale_cov	All covariances multiplied by 2.5	Cluster centres hold while the clusters spread and begin to overlap
reweight	Weights tilted linearly from 2.0 to 0.2, then renormalised	Configuration and shapes unchanged; the relative mass of the clusters shifts

Table 4: The four source-to-target relations. All are shared between the training and the test side.

Measured across the thirty-two test tasks, the ratio of mean between-component distance to within-component spread has a median of 8.1 (interquartile range 6.4 to 9.3), so the cluster structure is pronounced throughout the family.

2.3 The CIFAR-100 partition scheme

What follows describes the image partition, which is complete but not yet in training. Each training or testing unit — hereafter an *episode* — comprises **four mutually disjoint streams of samples**. The first principle of the split is not that images are partitioned but that **semantic tasks** are: all images of a given semantic class belong exclusively to the training set or exclusively to the test set. The purpose of this constraint is to separate genuine generalisation of the transfer rule from mere memorisation of particular task pairs.

Stream	Size	Drawn from		Content and purpose
Source support	500 (64/step)	source-role class	fine	Supplies the set encoder with the source coordinate. Several hundred images are used because the sampling noise of the pooled representation decays with the inverse square root of the sample count
Source query	100 (32/step)	source-role class	fine	Used solely to compute the source denoising loss; never enters coordinate inference
Target support	1/2/5/10/20	target-role class	fine	During training it supplies the target-only comparison coordinate; at deployment it is the sole evidence available for refinement. Nested across sizes
Target query	580 (32/step)	target-role class	fine	During training it carries the target and transport losses; at deployment it is reserved entirely for evaluation and may not enter refinement

Table 5: The four streams of an episode.

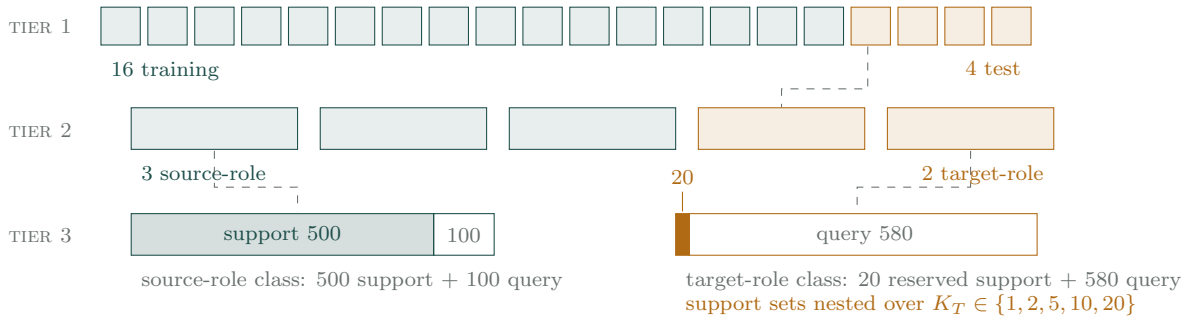


Figure 3: Tier 1: the twenty superclasses, split sixteen to four. Tier 2: the five fine classes of one superclass, three taking the source role and two the target role. Tier 3: the six hundred images of one fine class of each role. The partition narrows tier by tier, every boundary fixed deterministically by the seed. **A target-role fine class holds exactly as many images as a source-role one** — the scarcity is imposed by restricting the support set, not inherited from any imbalance in the dataset. This bears on how results should be read: the protocol tests scarcity of *support samples*, not rarity of the category itself.

Sixteen of the twenty superclasses go to training and four to test. Within each superclass, three of the five fine classes take the source role and retain their full image pool, while two take the target role. Since CIFAR-100 is itself perfectly balanced at six hundred images per fine

class, the scarcity of target images is **constructed** by this protocol rather than inherited from the data. The result is 96 training episodes over 48,000 images and 24 test episodes over 12,000.

Each tier draws on its own random stream derived from the global seed, so reconfiguring one tier leaves the others undisturbed. The partition stores only global image indices, occupying some 350 kB, and has been verified bit-for-bit identical under two NumPy versions.

2.4 Three departures from the specified protocol

The partition scheme was specified by the project and differs substantively from the one laid down in the source document. These differences are deliberate rather than oversights, but they alter how certain results should be read.

Clause	What is prescribed	What is done	Consequence
§12.3	Unaltered images as source; target constructed by deterministic corruption. Source and target are two domains of <i>the same</i> semantic category	Source and target are two <i>different</i> fine classes within one superclass	The relation changes from a domain shift into a difference between sibling categories, and collapses the relation descriptor to a single relation
§6.2	Semantic tasks divided three ways: training, validation, test	Validation removed; its superclasses folded into the test set	No legitimate venue remains for hyperparameter selection. All hyperparameters retain the document’s defaults and no search has been performed
§A.1	The episode tuple carries domain identifiers d_S, d_T	Not stored	Source and target are <i>roles</i> rather than <i>domains</i> ; nothing to store

Table 6: Departures from the source document’s partition protocol.

The first departure deserves expansion. The central proposition of the source document is that the *change* between related distributions is low-dimensional, and relatedness is there guaranteed by holding the semantic category fixed across two domains. Once sibling fine classes are substituted, source and target share only a superclass, and their relatedness falls appreciably. Section 6 quantifies this effect: as the relatedness of the task family declines, the achievable accuracy captured by the low-dimensional coordinate falls from 58% to 22%. The present partition therefore sits towards the unfavourable end of that curve.

These clauses have been encoded as a repeatable conformance check with twenty-one specification assertions, the three departures being reported separately and excluded from the failure count.

3 Data flow in both phases

Training and deployment employ the same modules, yet the destinations of the four streams differ sharply. One rule summarises training: **support sets enter only the encoder, query sets enter only the losses**. Deployment imposes a second and stricter rule: **the target query set may take no part in any optimisation**.

Stream	During training	At deployment
Source support	Set encoder $\rightarrow$ source coordinate	Set encoder $\rightarrow$ source coordinate (identical)
Source query	Noising $\rightarrow$ backbone and heads $\rightarrow$ source denoising loss	Not used
Target support	Set encoder $\rightarrow$ target-encoded coordinate (comparison branch)	Sole evidence for refinement
Target query	Noising $\rightarrow$ backbone and heads $\rightarrow$ target and transport losses	Evaluation only

Table 7: Where each stream goes in each phase.

3.1 Training

Three coordinates are produced by the same encoder — two by encoding samples, one by transport prediction:

$$z_S = \rho_\psi\left(\frac{1}{m} \sum_i h_\psi(x_i)\right), \quad x_i \in D_{y,S}^s, \quad (2)$$

$$z_T^{\text{enc}} = \rho_\psi\left(\frac{1}{m} \sum_i h_\psi(x_i)\right), \quad x_i \in D_{y,T}^s, \quad (3)$$

$$\tilde{z}_T = z_S + \Delta_\gamma(z_S, c_{S \rightarrow T}). \quad (4)$$

The query sets are then noised and passed forward:

$$x_t = \alpha_t x_0 + \sigma_t \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad t \sim \text{Unif}\{0, \dots, T-1\}, \quad (5)$$

$$H = \text{backbone}(x_t, t), \quad \hat{e}_0 = \text{baseline head}(H), \quad R = \text{basis head}(H), \quad (6)$$

$$\hat{e}_z = \hat{e}_0 + \sum_{\ell=1}^k z_\ell R_\ell. \quad (7)$$

Two aspects of the fusion deserve comment. First, **the two target-side losses share a single noising operation** — the same timesteps and the same Gaussian noise — so they form a paired comparison and the variance of that comparison is substantially reduced. Second, **they share a single backbone pass**: $\hat{e}_0$ and R are each computed once, then contracted against two different coordinates. This halves the backbone computation at no cost in precision.

Concretely, the coordinate of shape (B, k) is contracted with the basis of shape (B, k, C, H, W) along the k axis and added elementwise to $\hat{e}_0$. The score field follows as $s = -\hat{e}/\sigma_t$; since the two differ only by a known time-dependent scale, superposing a basis in noise-prediction space is equivalent to superposing a basis in score space.

3.2 Deployment

Confronted with a semantic task never seen during training, all network parameters are frozen:

1. $z_S = \rho_\psi(\text{mean } h_\psi(\text{source support}))$ — may be accumulated in a stream;
2. $\tilde{z}_T = z_S + \Delta_\gamma(z_S, c)$ — no target sample observed;
3. $z_T^* = \arg \min_z \{\text{denoising loss on target support} + \text{prior}\}$ — only z varies;
4. reverse sampling $x_T \sim \mathcal{N}(0, I) \rightarrow \dots \rightarrow x_0$, calling $\hat{e}_{z_T^*}$ at each step.

Step (1) warrants comment. Because mean pooling is linear, accumulating the sum of per-sample features together with a count across minibatches, and applying the read-out head once at the end, is numerically equivalent to feeding all samples at once — the measured discrepancy is of order 10^{-8} . This allows deployment to draw on thousands of source images without holding them in memory simultaneously. Training, which must retain gradients, subsamples instead.

The signature of the refinement function **does not include the target query set as a parameter**. This is deliberate: one of the protocol’s red lines is that deployment-time refinement must not use the target query set, and excluding it from the signature renders that violation inexpressible at the level of types.

4 Hyperparameters and module specifications

The task coordinate has dimension $k = 16$, the starting value recommended by the source document. It must be recorded that the analytic validation stage indicates this value is insufficient for the present task family: trained to convergence, $k = 16$ attains 38.4% of the achievable accuracy, whereas $k = 64$ attains 56.9% (Section 6).

Module	Structure	Params
Shared backbone (<i>image domain</i>)	Standard Denoising Diffusion Probabilistic Model U-Net. Base width 128 channels; channel multipliers (1, 2, 2); two residual blocks per level; self-attention at 16×16 . Descends to 8×8 , passes a middle block, ascends to 32×32	27.74 M
Shared backbone (<i>vector domain</i>)	Four-layer residual Multi-Layer Perceptron, hidden width 256, timestep injected by scale-and-shift modulation. Two-dimensional points are represented as a 1×1 spatial field, reusing the same backbone interface	1.22 M
Baseline head	Single 3×3 convolution, $128 \rightarrow 3$ channels	3,459
Basis head	Single 3×3 convolution, $128 \rightarrow 48$ channels ($= k \times C_{\text{img}}$), reshaped into k tensors of image shape	55,344
Set encoder h_ψ	Three stride-2 convolutions ($3 \rightarrow 64 \rightarrow 128 \rightarrow 128$), each followed by group normalisation and a Sigmoid Linear Unit; global average pool and a linear layer to 256 dimensions	0.327 M
Set encoder ρ_ψ	Two-layer perceptron, $256 \rightarrow 256 \rightarrow k$	(incl.)
Transport network	Residual Multi-Layer Perceptron $k \rightarrow 64 \rightarrow 64 \rightarrow k$ with Sigmoid Linear Unit activations; final layer initialised small so the map is near-identity at the outset	6,288

Table 8: Module specifications, exported from the running configuration.

All comparison methods **share one refinement budget object**, so step count, learning rate and prior strength are identical across them. This alignment is structural rather than conventional: the budget is passed to each strategy as a single object. A caution is nonetheless warranted. Where a comparison method differs by orders of magnitude in parameter count — full fine-tuning being the obvious case — an equal step count does not constitute a fair comparison; measurement confirms this in Section 6.

5 Loss functions

Meta-training optimises all network parameters and its loss carries four terms; meta-testing freezes all network parameters, optimises the coordinate alone, and its objective carries two. The denoising term is the same function throughout; only the coordinate used in the forward pass, and the sample set over which the expectation is taken, distinguish one instance from another.

Quantity	Value	Remark
Diffusion steps	1000	Cosine noise schedule
Loss weighting $w(t)$	1	The standard noise-prediction objective
Optimiser, learning rate	AdamW, 2×10^{-4}	500 steps linear warm-up; gradient norm clipped to 1
Exponential Moving Average decay	0.999	Evaluation and diagnostics use the averaged weights
Loss weights λ_T, λ_{tr}	1.0, 1.0	Recommended starting values
Coordinate regularisation λ_z	1×10^{-4}	Light penalty against degenerate coordinates
Source samples per step	64	Subsampled under memory constraint; deployment is not so limited
Query samples per step	32	On each of the source and target sides
Refinement steps J	25	Candidate range {0, 5, 10, 25, 50, 100}
Refinement learning rate η_z	1×10^{-2}	Candidate range $\{10^{-3}, 3 \times 10^{-3}, 10^{-2}, 3 \times 10^{-2}\}$
Prior strength β_0	1.0	Candidate range {0, 0.01, 0.1, 1, 10}

Table 9: Training and refinement hyperparameters.

5.1 Meta-training

$$\mathcal{L} = \mathcal{L}_{\text{src}} + \lambda_T \mathcal{L}_{\text{tgt}} + \lambda_{\text{tr}} \mathcal{L}_{\text{trans}} + \lambda_z \mathcal{L}_z \quad (8)$$

First term: source denoising loss. Forward pass with the source coordinate, expectation over the source query set:

$$\mathcal{L}_{\text{src}} = \mathbb{E}_{x_0 \in D_{y,S}^q, t, \epsilon} \left[w(t) \left\| \epsilon - \hat{\epsilon}_{\phi, z_S}(x_t, t) \right\|_2^2 \right]. \quad (9)$$

This term anchors the source coordinate. Without it the source coordinate is subject to no constraint and need not genuinely explain the source distribution, whereupon the mapping learned by the transport network loses its footing. The source side is also the most abundantly sampled, so it is chiefly this term that trains the shared backbone and basis.

Second term: target denoising loss. Forward pass with the target-encoded coordinate:

$$\mathcal{L}_{\text{tgt}} = \mathbb{E}_{x_0 \in D_{y,T}^q, t, \epsilon} \left[w(t) \left\| \epsilon - \hat{\epsilon}_{\phi, z_T^{\text{enc}}}(x_t, t) \right\|_2^2 \right]. \quad (10)$$

This trains the encoder to infer a coordinate directly from a *sparse* target support set. It serves both as a comparison baseline and as the fallback when no source data are available.

Third term: transport loss. Forward pass with the transported coordinate, over the *same* target query set:

$$\mathcal{L}_{\text{trans}} = \mathbb{E}_{x_0 \in D_{y,T}^q, t, \epsilon} \left[w(t) \left\| \epsilon - \hat{\epsilon}_{\phi, \tilde{z}_T}(x_t, t) \right\|_2^2 \right]. \quad (11)$$

The supervisory signal for the transport network is **denoising performance**, not agreement with the target-encoded coordinate. The distinction matters: a sparse target encoding is itself noisy, and compelling the transported result to approach it may be counterproductive; the optional alignment term is therefore disabled by default. Note further that the second and third terms act on the same batch of query samples, so a per-episode head-to-head comparison between target-only and source-informed adaptation is conducted throughout training.

Fourth term: coordinate regularisation. Applied only to the two *encoded* coordinates; the transported coordinate is excluded:

$$\mathcal{L}_z = \|z_S\|_2^2 + \|z_T^{\text{enc}}\|_2^2. \quad (12)$$

Its strength must be chosen with care. Set too weakly, the encoder may emit a near-constant vector of substantial magnitude, reducing the basis to extra capacity for the shared model rather than a task-specific modulation of it; set too strongly, the coordinate becomes discriminative but loses its influence.

5.2 Meta-testing

All network parameters are frozen and the coordinate is the only variable under optimisation:

$$z_T^* = \arg \min_z \left\{ \frac{1}{K_T} \sum_{i=1}^{K_T} \mathbb{E}_{t,\epsilon} \left[w(t) \|\epsilon - \hat{\epsilon}_{\phi,z}(x_{i,t}^T, t)\|_2^2 \right] + \frac{\beta_0}{K_T} \|z - \tilde{z}_T\|_2^2 \right\}. \quad (13)$$

The two divisions by the support size K_T are not redundant; they are the heart of the design. Dividing the denoising term by K_T keeps its gradient scale stable across support sizes. Dividing the prior term by the same quantity causes **the relative influence of the source-derived prior to decay naturally as target evidence accumulates**. In the extreme few-shot regime the prior dominates; once target samples are plentiful it recedes. Experiment confirms this: the advantage of transport over target-only adaptation is greatest at a support set of one image and falls exactly to zero by twenty.

The target-only comparison method uses the identical objective, replacing only the prior centre — the transported coordinate gives way to the target-encoded coordinate — so that optimisation budget and regularisation structure remain exactly aligned.

6 Results to date

This section reports the analytic validation stage, conducted on two-dimensional Gaussian mixtures. That stage was undertaken before CIFAR-100 for a specific reason: Gaussian noising preserves the Gaussian-mixture form, so the true score field of the target distribution admits a closed form. Model output can therefore be compared against *ground truth* directly, without recourse to the visual plausibility of generated samples or to any surrogate measure. Should the method fail here, the cause cannot be obscured by a backbone that has yet to learn images.

The **relative score-field error** is $\|\hat{s} - s^*\|/\|s^*\|$, evaluated at points drawn from the true noised distribution — that is, in the region the sampler will actually visit. The **sliced Wasserstein distance** measures the discrepancy between the generated point cloud and the true target distribution. Both are lower the better.

Four observations follow. *First*, the method leads clearly in the extreme few-shot regime. *Second*, target-only adaptation on a single image is worse than doing nothing at all (1.049 against 0.905): a coordinate inferred from one sample is noise rather than information. *Third*, the advantage disappears as the support set grows; by twenty images target-only has overtaken, and the difference is statistically indistinguishable. *Fourth*, full fine-tuning is not viable in this regime: under a matched budget it has barely moved, while under a generous budget it overfits catastrophically at low K_T — two thousand steps on a single image drive the error to 7.02, far worse than no adaptation whatever. This is not misconfiguration but the method’s genuine behaviour: 1.2 million parameters cannot be constrained by one sample.

Method	Role	$K_T = 1$		$K_T = 5$		$K_T = 20$	
		Err.	SW ₂	Err.	SW ₂	Err.	SW ₂
No adaptation ($z = 0$)	lower ref.	0.9045	0.9726	0.9049	0.9879	0.9055	0.9802
Target-only	baseline	1.0491	0.8971	0.8318	0.6597	0.7052	0.5244
Direct source reuse	ablation	0.8228	0.7454	0.8045	0.6352	0.7551	0.6795
Transport, no refinement	ablation	0.8060	0.7216	0.8028	0.7233	0.8103	0.7046
Transport + refinement	ours	0.7774	0.6885	0.7515	0.6680	0.7186	0.6002
Full fine-tuning, $J = 25$	comparison	0.9141	0.8927	0.8507	0.6629	0.8273	0.5798
Full fine-tuning, $J = 2000$	comparison	7.0156	1.5344	5.3198	0.7342	1.4794	0.5702
Abundant-target oracle	upper ref.	0.6612	0.5552	0.6649	0.5273	0.6617	0.5215

Table 10: Comparison of methods, $k = 16$, 80,000 training steps, means over twelve held-out tasks. *Ours* is the complete algorithm; *ablation* removes one of its components; *baseline* is an independent comparison method; *reference* rows are not usable methods and serve only to bound the attainable range.

6.1 Statistical significance

To establish whether the differences are resolvable, the comparison was repeated over thirty-two held-out tasks with three random seeds, giving ninety-six paired samples; pairing removes the large between-task variance.

Comparison	$K_T = 1$	$K_T = 5$	$K_T = 20$
Ours – target-only	$+0.2374 \pm 0.034^*$	$+0.0852 \pm 0.021^*$	-0.0010 ± 0.010
Ours – direct source reuse	$+0.0863 \pm 0.029^*$	$+0.0642 \pm 0.021^*$	$+0.0597 \pm 0.020^*$
Ours – transport without refine.	$+0.0083 \pm 0.007^*$	$+0.0459 \pm 0.014^*$	$+0.0818 \pm 0.013^*$
Ours – no adaptation	$+0.1528 \pm 0.027^*$	$+0.1916 \pm 0.022^*$	$+0.2256 \pm 0.022^*$

Table 11: Paired differences with 95% confidence intervals; positive favours the present method. * marks intervals excluding zero.

Every interval excludes zero save the comparison against target-only at $K_T = 20$, which reads -0.0010 ± 0.0104 and falls exactly on zero — the advantage does not decay slowly towards some small residue but vanishes precisely, in accordance with the document’s statement that the method should converge to target-only behaviour as target evidence grows. A second point deserves emphasis: **the advantage over direct source reuse does not decay with support size**, deriving as it does from the relation the transport network has learned rather than from the number of target samples available. Measurement in coordinate space agrees: taking the optimal coordinate from abundant target data as reference, the relative errors $\|z - z^*\|/\|z^*\|$ are 0.754 for the transported prediction, 0.866 for direct source reuse, 0.844 for target-only and 1.000 for the zero coordinate.

6.2 Quantitative test of the low-dimensionality hypothesis

The document’s first claim is quantified through a *captured fraction*,

$$\text{captured} = \frac{\text{no-adaptation error} - \text{oracle error}}{\text{no-adaptation error} - \text{converged full-fine-tuning error}}, \quad (14)$$

the share of the total achievable improvement that the low-dimensional coordinate secures. The document’s criterion is that this should constitute “most” of what is achievable.

Neither factor alone suffices. The document’s suggested $k = 16$ attains only 38.4% even trained to convergence; only at $k = 64$, fully trained, does it reach 56.7%, at which point it has levelled off (a rise of 1.1 percentage points between 160k and 320k steps). Their convergence

Configuration	No adapt.	Oracle	Conv. full FT	Captured
$k = 16$, 80k steps	0.8821	0.6320	0.1276	33.2%
$k = 16$, 320k steps	0.8741	0.5965	0.1508	38.4%
$k = 64$, 80k steps	0.8868	0.5396	0.1267	45.7%
$k = 64$, 320k steps	0.8779	0.4690	0.1565	56.7%

Table 12: The captured fraction depends on both coordinate dimension and training sufficiency.

behaviour also differs: $k = 64$ gains 11.2 points from a fourfold increase in training and is therefore constrained chiefly by *training budget*; $k = 16$ gains only 4.6, having nearly converged by 80k steps, and is constrained by *capacity*. This answers the instruction to “start with $k = 16$ and test 64 only if the curve has not saturated”: on this task family the curve has not saturated, so 64 is not optional but required. It should be added that $k = 64$ already amounts to 2.8 times the intrinsic degrees of freedom of the task — a four-component two-dimensional Gaussian mixture has twenty-three free parameters — and that in the image domain no such intrinsic dimension is definable, so the ratio does not extrapolate.

6.3 The effect of task-family relatedness

The document confines its proposition to what holds “within a related family of tasks”. To quantify the weight that qualifier carries, a family was constructed whose members are perturbations of a common template mixture at adjustable strength. All configurations use $k = 16$ and 80k steps, so the comparison is internally consistent.

Task family	No adapt.	Oracle	Full FT	Captured
Perturbation 0.1 (closely related)	0.7513	0.3835	0.1179	58.1%
Perturbation 0.25	0.8910	0.5583	0.1220	43.3%
Perturbation 0.5	0.9516	0.6949	0.1359	31.5%
Perturbation 1.0 (nearly unrelated)	0.9679	0.7808	0.1239	22.2%

Table 13: The captured fraction falls monotonically with relatedness.

At identical $k = 16$ and identical training, **substituting a closely related task family yields a greater improvement than raising the coordinate dimension from sixteen to sixty-four** (33.7% to 45.7%). For this task family, how closely the tasks are related is the more decisive factor of the two. This bears directly on the partition scheme of Section 2.4, which takes sibling fine classes as source and target and therefore sits towards the unfavourable end of this table.

6.4 Structural diagnostics

The minimal structural diagnostics exist to exclude one possibility: that performance is attributed to a coordinate mechanism the network has in fact ignored.

6.5 Three methodological cautions

Each was encountered and corrected during this stage. All three share one root: **when the methods being compared differ by orders of magnitude in parameter count, a fixed-budget comparison measures the behaviour of the optimiser rather than of the method.**

1. **Matched-budget full fine-tuning confers a spurious advantage.** At twenty-five steps apiece, full fine-tuning gives 0.914 and the coordinate method 0.777, which appears

Diagnostic	Value	Reading
Basis usage $\ \epsilon_{\text{res}}\ /\ \epsilon_{\text{base}}\ $	1.32	The basis is heavily used; a value near zero would indicate the coordinate is ignored
Gain of correct coordinate over zero	+0.107	The coordinate carries usable information
Gain of correct coordinate over one shuffled across tasks	+0.300	Coordinates correspond to their tasks; any non-zero vector will not do
Relative spread of coordinates across tasks	1.35	No collapse; a value near zero would indicate a near-constant encoder output

Table 14: Structural diagnostics, unrelated task family, $k = 16$, twenty-four held-out tasks.

decisive; yet full fine-tuning reaches 0.13 on convergence. The whole of the difference arises from a mismatch in learning-rate scale. Any report of this comparison must state the converged value alongside it.

2. **The upper-bound reference itself overfits.** Trained too long on a finite target sample, full fine-tuning overfits; and because the score-field error is measured against ground truth, this manifests as the error rising again — one checkpoint gave 0.160, 0.169 and 0.200 at two thousand, eight thousand and twenty thousand steps. The upper bound must be taken as the minimum over a budget grid, with the spread reported.
3. **Evaluation over few tasks cannot resolve effects of five per cent.** Two evaluations of the same configuration once returned +0.034 and -0.048 for the comparison against direct source reuse — opposite conclusions. Under paired statistics over thirty-two tasks and three seeds the true value is $+0.086 \pm 0.029$, stably positive.

Code: `code/python/meta-diffusion/` — training entry point `runner/train.py`, analytic validation entry point `runner/stage1_gmm.py`. Parameter counts and hyperparameter values were exported from the running configuration. The upper-bound reference is the minimum over the budget grid $\{J = 500, 2000, 8000\} \times \{\eta = 3 \times 10^{-4}, 10^{-4}\}$. The analytic ground truth was cross-validated by three independent methods: automatic differentiation (3.6×10^{-13}), finite differences (1.3×10^{-10}), and a four-hundred-thousand-sample Monte Carlo check of the noised marginal.

update 09/09/2026

Meta-Diffusion: Experimental Record

Abstract

This document records the framework, the data protocol, the loss functions, every experiment run to date, and a step-by-step procedure written so that the image experiments can be reproduced exactly. Every figure quoted here was exported from the running code rather than recalled. The test split has not been used for any choice at any point in the programme.

7 Framework structure

Four modules, organised by one principle: everything high-dimensional is shared across all tasks and frozen at deployment; everything task-specific is compressed into the coordinate $z \in \mathbb{R}^k$. The noise prediction is a sum—a shared baseline plus a weighted combination of learned perturbation directions.

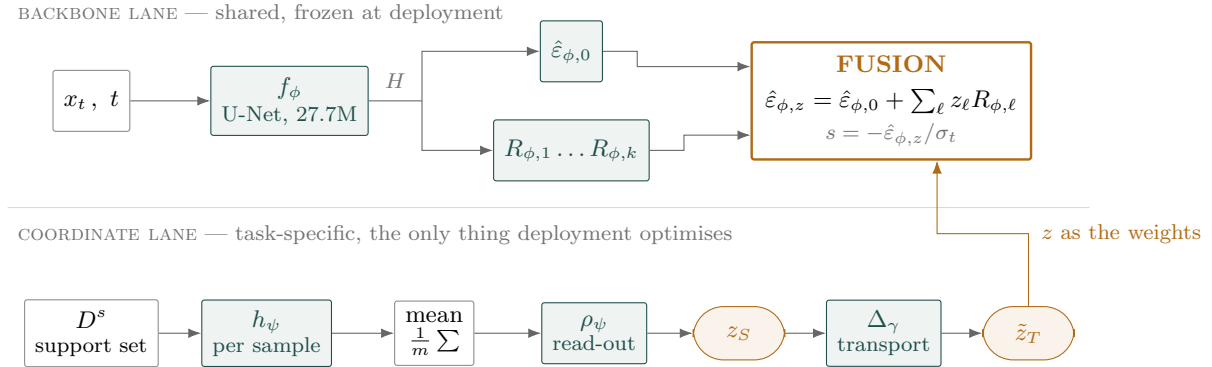


Figure 4: Architecture. Green marks the shared parameters ϕ and ψ , frozen at deployment; amber marks the task coordinate, the only object deployment optimises—sixteen scalars against 27.8 million frozen weights. One pass of f_ϕ feeds both heads, which is what makes this one network rather than k separate denoisers. The transport map Δ_γ receives z_S and the relation descriptor only; no target sample reaches it, and that constraint is the proposition under test.

8 Dataset construction

CIFAR-100 is a two-level taxonomy: twenty superclasses, each holding exactly five fine classes, each fine class holding exactly 600 images of 32×32 pixels in colour.

The split divides **semantic tasks, not images**. A fine class belongs entirely to one of the three ways, and the division is made at the superclass level so that near-identical siblings cannot straddle it.

8.1 Within one fine class

Each class’s 600 images are shuffled once by a seed-derived permutation and cut into two disjoint pools, then each pool is cut again into support and query.

8.2 How the target domain is made

The target domain is produced by applying a fixed, deterministic, semantics-preserving image transformation to the target pool. Three are implemented, in the style of the CIFAR-100-C benchmark, each at five severities; severity 3 is the default.

Symbol	Module	Params	Definition and constraint
f_ϕ	shared backbone	27.74 M	Maps (x_t, t) to a feature tensor H at input resolution. Carries no noise-prediction head of its own
$\hat{\varepsilon}_{\phi,0}$	baseline head	3,459	1×1 convolution, $C_{\text{feat}} \rightarrow C_{\text{img}}$. The task-independent noise prediction
$R_{\phi,1..k}$	basis head	55,344	Convolution to $k C_{\text{img}}$ channels, reshaped to k image-shaped directions. Initialised small but non-zero
h_ψ, ρ_ψ	set encoder	0.33 M	Permutation-invariant map from a set to a co-ordinate. Source and target share this one encoder
Δ_γ	transport map	6,288	Residual multi-layer perceptron (MLP), $(k + d_c) \rightarrow 64 \rightarrow 64 \rightarrow k$, operating purely in coordinate space
z	task coordinate	16	The sole degree of freedom at deployment

Table 15: Module inventory at $k = 16$, three image channels, 32×32 pixels. ϕ denotes backbone and both heads; ψ the encoder; γ the transport map.

	SUPERCLASS	ITS FIVE FINE CLASSES — EACH ONE IS A SEPARATE TASK
TRAIN STANDARD	aquatic mammals	beaver · dolphin · otter · seal · whale
	fish	aquarium fish · flatfish · ray · shark · trout
	fruit and vegetables	apple · mushroom · orange · pear · sweet pepper
	household electrical devices	clock · keyboard · lamp · telephone · television
	household furniture	bed · chair · couch · table · wardrobe
	large man-made outdoor things	bridge · castle · house · road · skyscraper
	large natural outdoor scenes	cloud · forest · mountain · plain · sea
	large omnivores and herbivores	camel · cattle · chimpanzee · elephant · kangaroo
	medium mammals	fox · porcupine · possum · raccoon · skunk
	reptiles	crocodile · dinosaur · lizard · snake · turtle
VALIDATION every choice	trees	maple tree · oak tree · palm tree · pine tree · willow tree
	vehicles 1	bicycle · bus · motorcycle · pickup truck · train
	food containers	bottle · bowl · can · cup · plate
	insects	bee · beetle · butterfly · caterpillar · cockroach
TEST final only	large carnivores	bear · leopard · lion · tiger · wolf
	vehicles 2	lawn mower · rocket · streetcar · tank · tractor
	flowers	orchid · poppy · rose · sunflower · tulip
	non-insect invertebrates	crab · lobster · snail · spider · worm
	people	baby · boy · girl · man · woman
	small mammals	hamster · mouse · rabbit · shrew · squirrel

Figure 5: The three-way split in full. Twelve superclasses and their sixty fine classes train the model; four superclasses and twenty fine classes carry every hyperparameter choice; four superclasses and twenty fine classes are reserved for the final number. Splitting by superclass rather than by fine class keeps near-identical siblings such as maple and oak on the same side: the test set contains no close relative of anything seen in training.

Stream	Symbol	Images	Pixels	Drawn per episode
source support	$D_{y,S}^s$	300	clean	M_S sampled without replacement
source query	$D_{y,S}^q$	50	clean	32 sampled without replacement
target support	$D_{y,T}^s$	20 reserved	transformed	the first K_T , so support sets are nested in K_T
target query	$D_{y,T}^q$	230	transformed	32 sampled without replacement

Table 16: Source pool 350 images, target pool 250, verified pairwise disjoint by global image identifier: 600 identifiers, 600 unique. Source and target therefore use *different underlying images*—pairing a clean image with its own transformed copy would leak, and real domain shift does not arrive in matched pairs.

Transformation	Operation	Parameter, severity 1 . . . 5	mean $ \Delta \mathbf{px} $
Gaussian blur	separable Gaussian convolution	$\sigma = 0.5, 0.75, 1.0, 1.5, 2.0$	0.085
Additive noise	seeded per image identifier	std = 0.08, 0.12, 0.18, 0.26, 0.38	0.133
Contrast	shrink toward each image’s own mean	factor = 0.75, 0.6, 0.45, 0.3, 0.2	0.184

Table 17: Change measured on the $[-1, 1]$ pixel scale at severity 3. Every transformation is reproducible from the triple (image identifier, type, severity); the additive-noise transformation derives its random seed from the image identifier, without which the “same” target domain would differ between runs.

9 Data flow

Support sets flow into the encoder and produce coordinates. Query sets flow into the diffusion forward process and produce losses. No stream does both.

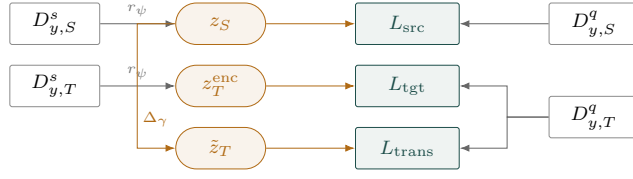
Two operations, easily conflated. One of the three transformations is itself called Gaussian noise, so the two are kept apart throughout this document:

- the **image transformation**—blur, additive noise or contrast reduction—applied when a target-stream image is fetched. It defines *what the target domain is*, and the model must learn to generate images that carry it;
- the **diffusion forward process** $x_t = \alpha_t x_0 + \sigma_t \varepsilon$, applied to query batches during training. It is part of the *training mechanism* and applies equally to source and target.

Stream	transformation defines the target domain	forward process the training mechanism	Why
$D_{y,S}^s$	no	no	source domain, and support sets only produce coordinates
$D_{y,S}^q$	no	yes	source domain, but a query set carries a denoising loss
$D_{y,T}^s$	yes	no	target domain, but a support set only produces a coordinate
$D_{y,T}^q$	yes	yes	target domain and a query set: both apply

Table 18: Read down the first column for “which domain is this”, and down the second for “does this stream carry a loss”. The two questions are independent, which is why the answer is a 2×2 rather than a single split.

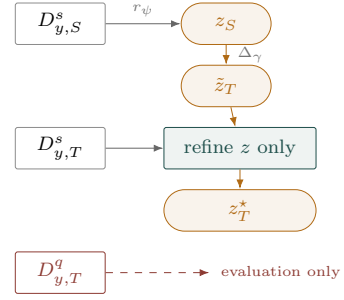
TRAINING



q_{sample} touches D^q only, never a support set.

L_{tgt} and L_{trans} share one forward-process draw on the same $D_{y,T}^q$.

DEPLOYMENT



ϕ, ψ, γ all frozen. Only the 16 scalars of z move.
 $D_{y,T}^s$ is the sole target evidence available.

Figure 6: Where each stream goes. The structural difference between the two halves is which arrows exist: training computes three losses and every module receives gradient; deployment freezes all parameters and refines the coordinate alone. The dashed path is the protocol’s red line—target query data reaches evaluation and nothing else, enforced at the type level, since the refinement function has no parameter through which it could arrive.

10 Loss functions

10.1 Notation

Write y for a semantic task, $t \sim \mathcal{U}\{0, \dots, T-1\}$ for a diffusion timestep and $\varepsilon \sim \mathcal{N}(0, I)$ for the noise. The forward process is

$$x_t = \alpha_t x_0 + \sigma_t \varepsilon, \quad \alpha_t^2 + \sigma_t^2 = 1, \quad (15)$$

and the weighting $w(t) = 1$ throughout—the plain ε -prediction objective.

10.2 The three coordinates

$$z_S = \rho_{\psi} \left(\frac{1}{|D_{y,S}^s|} \sum_{x \in D_{y,S}^s} h_{\psi}(x) \right) \quad (16)$$

$$z_T^{\text{enc}} = \rho_{\psi} \left(\frac{1}{|D_{y,T}^s|} \sum_{x \in D_{y,T}^s} h_{\psi}(x) \right) \quad (17)$$

$$\tilde{z}_T = z_S + \Delta_{\gamma}(z_S, c_{S \rightarrow T}) \quad (18)$$

The third line takes z_S and the relation descriptor and nothing else. It contains no target sample.

10.3 The predictor each loss is built on

Every term below evaluates the same predictor at a different coordinate:

$$\hat{\varepsilon}_{\phi,z}(x_t, t) = \hat{\varepsilon}_{\phi,0}(x_t, t) + \sum_{\ell=1}^k z_{\ell} R_{\phi,\ell}(x_t, t) \quad (19)$$

10.4 Term 1 — source denoising loss

Trains the backbone and the baseline head on the source domain, and is the only signal that shapes z_S through the encoder.

$$L_{\text{src}} = \mathbb{E}_{x_0 \sim D_{y,S}^q} \mathbb{E}_t \mathbb{E}_\varepsilon \left[w(t) \left\| \varepsilon - \hat{\varepsilon}_{\phi, z_S}(\alpha_t x_0 + \sigma_t \varepsilon, t) \right\|_2^2 \right] \quad (20)$$

10.5 Term 2 — target denoising loss, encoder coordinate

The same functional on target-domain query images, using the coordinate encoded from the K_T target support images. This is what teaches the encoder to produce meaningful coordinates for a target set; without it the coordinate space would be shaped by source data alone.

$$L_{\text{tgt}} = \mathbb{E}_{x_0 \sim D_{y,T}^q} \mathbb{E}_t \mathbb{E}_\varepsilon \left[w(t) \left\| \varepsilon - \hat{\varepsilon}_{\phi, z_T^{\text{enc}}}(\alpha_t x_0 + \sigma_t \varepsilon, t) \right\|_2^2 \right] \quad (21)$$

10.6 Term 3 — transport denoising loss

Identical to term 2 but for the coordinate: the transported one, predicted from source data alone. This is the only term that trains Δ_γ .

$$L_{\text{trans}} = \mathbb{E}_{x_0 \sim D_{y,T}^q} \mathbb{E}_t \mathbb{E}_\varepsilon \left[w(t) \left\| \varepsilon - \hat{\varepsilon}_{\phi, \tilde{z}_T}(\alpha_t x_0 + \sigma_t \varepsilon, t) \right\|_2^2 \right] \quad (22)$$

10.7 Term 4 — coordinate regulariser

A light penalty preventing the encoder from drifting to coordinates of large magnitude. Set too weakly, the encoder may emit a near-constant vector of substantial size, turning the basis into extra shared capacity; set too strongly, the coordinate loses its influence.

$$L_z = \|z_S\|_2^2 + \|z_T^{\text{enc}}\|_2^2 \quad (23)$$

Note that $\tilde{z}_T$ is absent, since penalising it would penalise Δ_γ for moving.

10.8 The total

$$\boxed{L_{\text{meta}} = L_{\text{src}} + \lambda_T L_{\text{tgt}} + \lambda_{\text{tr}} L_{\text{trans}} + \lambda_z L_z} \quad \lambda_T = \lambda_{\text{tr}} = 1, \quad \lambda_z = 10^{-4} \quad (24)$$

10.9 What is actually computed each step

The expectations above are estimated by a single Monte-Carlo draw per query image. With a batch of $B = 32$ query images, one timestep and one noise vector drawn independently per image, and $C \cdot H \cdot W = 3 \cdot 32 \cdot 32$ pixels:

$$t_i \sim \mathcal{U}\{0, \dots, 999\}, \quad \varepsilon_i \sim \mathcal{N}(0, I), \quad x_{i,t} = \alpha_{t_i} x_i + \sigma_{t_i} \varepsilon_i \quad (25)$$

$$\hat{L}(D, z) = \frac{1}{B} \sum_{i=1}^B w(t_i) \left[\frac{1}{C H W} \sum_{\text{pixels}} (\varepsilon_i - \hat{\varepsilon}_{\phi, z}(x_{i,t}, t_i))^2 \right] \quad (26)$$

Two details matter for reproduction. The inner reduction is a **mean over pixels, not a sum**, so the loss scale is independent of image resolution. And $w(t) = 1$ throughout; signal-to-noise and truncated signal-to-noise weightings are implemented but were not used in any experiment reported here.

Terms 2 and 3 share their random draws. One batch $\{x_0\}$, one set of timesteps $\{t_i\}$, one set of noise vectors $\{\varepsilon_i\}$, and one backbone pass serve both. Their difference is therefore a paired estimate of what the transport map adds over encoding the target directly, with the sampling noise common to both cancelling out.

10.10 The deployment objective

At deployment ϕ, ψ, γ are frozen and a single coordinate is fitted on the K_T target support images:

$$z_T^* = \arg \min_z \left\{ \frac{1}{K_T} \sum_{i=1}^{K_T} \mathbb{E}_{t, \varepsilon} \left[w(t) \|\varepsilon - \hat{\varepsilon}_{\phi, z}(x_{i,t}^T, t)\|_2^2 \right] + \frac{\beta_0}{K_T} \|z - \tilde{z}_T\|_2^2 \right\} \quad (27)$$

with $J = 25$ steps, $\eta_z = 10^{-2}$, $\beta_0 = 1$, and 16 (t, ε) draws per step.

Both divisions by K_T matter. Dividing the denoising term holds the gradient scale steady across support sizes; dividing the prior term by the same quantity makes the source-derived prior’s relative influence decay as target evidence accumulates—it dominates at one sample and recedes once samples are plentiful.

11 Experiments run

11.1 What the numbers in these tables are

Seven quantities appear across the tables below. Two exist only on Gaussian mixtures, where the true score function is available in closed form; the rest apply to the image experiments as well.

Quantity	Definition	Units, and which way is better
relative score-field error	$\sqrt{\sum \ \hat{s} - s^*\ ^2 / \sum \ s^*\ ^2}$, accumulated over five timesteps at 2048 points each, drawn from the true noised distribution $x_t \sim p_t$ rather than a uniform grid	dimensionless; 0 is exact, 1 is as wrong as predicting zero. Lower is better
sliced Wasserstein-2	Project both point clouds onto 256 random unit directions, sort, take the root-mean-square difference	data units. Lower is better
denoising loss	$\mathbb{E}[w(t)\ \varepsilon - \hat{\varepsilon}\ ^2]$ on held-out target query images. The training objective itself, on data never fitted	squared-noise units; typical converged values 0.02–0.15. Lower is better
gain vs $z = 0$	$\text{loss}(z=0) - \text{loss}(\text{correct } z)$	same units as the loss. Higher <i>looks</i> better—see E9
basis usage r_{basis}	$\ \sum_{\ell} z_{\ell} R_{\ell}\ / (\ \hat{\varepsilon}_0\ + \epsilon)$, per sample	dimensionless; 0 means the basis is unused
coordinate spread	mean pairwise distance between episode coordinates, divided by their mean norm	dimensionless; below 0.05 the encoder emits a near-constant
relative coordinate distance	$\ z - z_T^{\text{abund}}\ / \ z_T^{\text{abund}}\ $, reference encoded from 230 target images	dimensionless. Lower is better

Every $\pm$ figure is a 95% confidence interval on a **paired per-episode difference**, not a standard deviation across runs: both strategies are evaluated on the same episode with the same noise draw, the difference taken per episode, and the interval formed over episodes. Between-task variance is roughly an order of magnitude larger than the effects being measured, so an unpaired comparison over a handful of episodes returns an arbitrary sign.

11.2 E1 — analytic validation on two-dimensional Gaussian mixtures

$k = 16$, 80,000 steps, 12 unseen tasks. Mechanism validated.

Procedure. Each task is a four-component Gaussian mixture; the target is that mixture under a controlled relation (rotation, translation, covariance scaling, reweighting) that recurs across tasks. Gaussian noising preserves the mixture form, so the true score has a closed form and model output is compared against *truth* rather than sample plausibility. The backbone is a 1.3M-parameter residual MLP; every other module and the entire training loop are unchanged from the image path.

Strategy	$K_T = 1$		$K_T = 5$		$K_T = 20$	
	score err	SW ₂	score err	SW ₂	score err	SW ₂
no adaptation ($z = 0$)	0.9045	0.9726	0.9049	0.9879	0.9055	0.9802
target only	1.0478	0.8966	0.8393	0.6595	0.7086	0.5336
reuse z_S	0.8243	0.7465	0.8010	0.6240	0.7586	0.6608
transport, no refine	0.8060	0.7216	0.8028	0.7233	0.8103	0.7046
transport + refine	0.7787	0.6951	0.7547	0.6717	0.7160	0.6117
full fine-tuning, $J = 25$	0.9169	0.8991	0.8591	0.6864	0.8330	0.5767
full fine-tuning, $J = 2000$	6.9697	1.5356	5.3606	0.7293	1.5218	0.3942
abundant-target oracle	0.6614	0.5561	0.6632	0.5289	0.6640	0.5203

Table 19: Relative score-field error and sliced Wasserstein-2 distance. Both lower-is-better.

Transport minus...	paired difference in relative score-field error		
	$K_T = 1$	$K_T = 5$	$K_T = 20$
no adaptation	$+0.1529 \pm 0.0273$	$+0.1913 \pm 0.0221$	$+0.2263 \pm 0.0217$
target only	$+0.2397 \pm 0.0338$	$+0.0861 \pm 0.0210$	$+0.0015 \pm 0.0106$
reuse z_S	$+0.0859 \pm 0.0287$	$+0.0611 \pm 0.0214$	$+0.0626 \pm 0.0196$
transport without refinement	$+0.0078 \pm 0.0072$	$+0.0473 \pm 0.0136$	$+0.0831 \pm 0.0130$

Table 20: 32 unseen tasks $\times$ 3 seeds, paired. Every interval excludes zero except transport against target-only at $K_T = 20$, where the advantage has correctly vanished. The margin being largest at one sample and gone by twenty is the shape the method predicts, and this is the strongest positive evidence in the programme.

11.3 E2 — CIFAR-100, sibling fine classes

$k = 16$, 6,000 steps, stopped early. Coordinate collapsed.

Procedure. Source and target are two *different* fine classes of one superclass—beaver and dolphin, say. Since CIFAR-100 is balanced, the target’s scarcity is imposed by restricting its support set rather than inherited from the data.

Result. The coordinate collapses to a near-constant: relative spread across episodes stays at 0.013–0.026 against a 0.05 threshold, flat over six diagnostic points while the denoising loss plateaued. Dissecting the step-5000 checkpoint located the cause: pooled encoder features have a cross-episode spread of only 0.058, and the source-to-target distance (0.067) is indistinguishable from it—one flower species reads much like another. Adding second-moment pooling did not fix it. Retained as a low-relatedness stress test.

11.4 E3 — CIFAR-100, one fixed transformation

$k = 16$, 50,000 steps, Gaussian blur at severity 3. Benefit decays to zero.

Procedure. The main protocol: source is a class’s clean images, target is different images of the same class under Gaussian blur. Diagnostics every 1000 steps on held-out classes.

Result. Collapse is solved—coordinate spread rises to 0.32 and source is cleanly separated from target (1.45). But the benefit vanishes over training:

step	1000	2000	5000	10000	20000	50000
gain vs $z = 0$	0.083	0.084	0.058	0.019	0.001	0.0002
basis usage	0.254	0.402	0.317	0.161	0.032	0.012

Two quantities move in opposite directions: the encoder keeps improving while the basis is progressively abandoned.

11.5 E4 — CIFAR-100, three transformations

$k = 16$, 50,000 steps, blur, additive noise and contrast. Same decay.

Procedure. Identical to E3 except that each episode draws one of three transformations, so the relation descriptor is active and the backbone cannot know which target domain applies without the coordinate. The relation is indexed by transformation type, which is legitimate because transformations recur across all three splits.

Result. The hypothesis failed. Gain peaks at 0.078 and decays to 0.0016; basis usage $0.29 \rightarrow 0.039$; coordinate spread meanwhile climbs to 1.21.

Strategy	denoising loss on held-out target query, lower is better		
	$K_T = 1$	$K_T = 5$	$K_T = 20$
no adaptation ($z = 0$)	0.0902	0.0902	0.0902
target support only	0.0888	0.0887	0.0887
reuse z_S	0.0901	0.0890	0.0888
transport, no refine	0.0887	0.0887	0.0887
transport + refine	0.0887	0.0887	0.0887
oracle (fits on target query)	0.0887	0.0887	0.0887

Table 21: Any non-zero coordinate beats $z = 0$ by 0.0015 ± 0.0002 —statistically real, 1.7% of the loss. No coordinate beats any other: transport, refinement and even the oracle upper bound land on the same number, and K_T changes nothing.

11.6 E5 — coordinate dimension sweep

$k \in \{16, 32, 64\}$, 20,000 steps each, validation classes. k is not the bottleneck.

Procedure. Three separate training runs, identical but for k , which changes the basis head’s output channel count and therefore requires retraining.

11.7 E6 — backbone capacity sweep

$k = 32$ fixed, base channels $\in \{128, 64, 32\}$, 20,000 steps. Read as success; later overturned.

k	gain vs $z = 0$ at its peak		gain vs $z = 0$	basis usage
	value	step	at 20k steps	at 20k steps
16	0.0839	2000	0.0007	0.0286
32	0.1050	1000	0.0005	0.0244
64	0.1288	1000	0.0004	0.0203

Table 22: Quadrupling the coordinate dimension changes nothing at convergence. A larger k raises the early peak and then decays slightly further.

Procedure. Coordinate dimension held at 32 while the backbone shrinks. The 64-channel run differs from the 128 only in width—same depth, same attention—so that comparison is clean; the 32-channel run additionally drops one downsampling level and attention, making it a larger step rather than a pure width change.

backbone width	basis usage	gain vs $z = 0$
base channels	at 20k steps	at 20k steps
128	0.024	0.0005
64	0.605	0.144
32	1.289	0.363

Table 23: A 53-fold rise in basis usage and 725-fold in apparent gain. Read at the time as the mechanism engaging once the backbone stops absorbing the domain difference. E9 shows this reading was wrong.

11.8 E7 — convergence of the small backbones

$k = 32$, base channels 32 and 64, 60,000 steps. The decay is universal.

Procedure. Every curve in E6 was still falling at 20,000 steps, so both small backbones were run to 60,000 and an exponential fitted to the second half of training.

backbone width	decay half-life	gain vs $z = 0$	gain vs $z = 0$
base channels	fitted, steps	at 60k steps	extrapolated to 200k
128	1,965	0.0005	1.3×10^{-31}
64	10,160	0.0097	6.9×10^{-7}
32	25,380	0.1192	2.6×10^{-3}

Table 24: Exponential decay toward zero in all three cases, not toward a positive asymptote. Capacity multiplies the half-life thirteen-fold but does not stop the decay. The denoising loss stays flat throughout, so this is not undertraining.

11.9 E8 — target-support and source-evidence sweeps

32-channel checkpoint at 60,000 steps, 32 validation episodes per point.

Procedure. Evaluation-only: both quantities are deployment-time and need no retraining. K_T swept over $\{1, 2, 5, 10, 20\}$ at the training M_S ; M_S swept over $\{16, 32, 64, 128, 256, 300\}$ at $K_T = 1$. Run on the small-backbone checkpoint because a collapsed one would measure nothing.

K_T	1	2	5	10	20	
transport – reuse	+0.0044	+0.0039	+0.0032	+0.0023	+0.0011	(± 0.0003)
M_S	16	32	64	128	256	300
loss (ours)	0.0286	0.0282	0.0279	0.0275	0.0269	0.0272

The transport advantage over plain source reuse behaves exactly as the method predicts—largest where target evidence is scarcest, shrinking as it accumulates. More source evidence helps monotonically. But target-only, transport and the oracle all report the same 0.0279 at every K_T , which E10 explains.

11.10 E9 — mean-coordinate substitution

*32-channel checkpoint, 24 validation episodes, paired. **Decisive.***

Procedure. Every earlier comparison measures the correct coordinate against $z = 0$, which answers “does the model need z to be non-zero” rather than “does the model use z to identify the task”. To separate them, replace each episode’s own coordinate with **one mean coordinate shared by every episode**—removing all task-specific content while keeping whatever the coordinates have in common—and evaluate all three on the same noised batch, the same timesteps, the same noise draw.

Coordinate fed to the fusion	denoising loss	What it shows
this episode’s own	0.0273	the method as intended
one shared mean, all episodes	0.0273	identical—no task-specific content lost
$z = 0$	0.1456	catastrophic—a needed constant deleted

Table 25: Zero percent of the coordinate’s benefit is task-specific. The shared mean has length 0.246 while each episode’s deviation from it averages 0.0169—the task-specific part is 6.9% of the coordinate’s length and worth nothing measurable. This re-reads E6: the rise from 0.0005 to 0.363 was the size of a constant offset, which a large backbone absorbs into its own weights and a small one leaves in the basis. Neither adapts.

11.11 E10 — loss sensitivity to coordinate error

32-channel checkpoint, 24 validation episodes.

Procedure. Displace the encoder’s target coordinate a given relative distance in a random direction and measure the denoising loss, to establish how precise a coordinate must be.

relative offset	0.00	0.20	0.40	0.80	1.60	3.00	$z = 0$
loss	0.0252	0.0254	0.0262	0.0294	0.0420	0.0842	0.1440

A wide flat basin: errors up to 0.2 cost essentially nothing. Every coordinate the system produces lands inside it, which is why transport, target-only and the oracle are indistinguishable. Note the anisotropy— $z = 0$ is a displacement of about 1.0 toward the origin and costs 0.1188, while a random displacement of 0.80 costs 0.0042, more than twenty times cheaper.

11.12 E11 — coordinate accuracy against an abundant reference

Single-transformation checkpoint at 50,000 steps, 40 validation episodes. Transport works in coordinate space.

Procedure. Encode 230 target images to obtain a well-estimated reference coordinate, then measure the distance to it before and after transport. Comparing against a K_T -sample encoding cannot do this, because at $K_T = 1$ the reference is itself noise.

K_T	relative distance to a reference from 230 target images			same distance for the K_T -sample encoding
	before transport	after transport	gap closed	
1	3.106 ± 0.073	0.227 ± 0.019	$+2.879 \pm 0.070$	0.397 ± 0.047
5	3.106 ± 0.073	0.227 ± 0.019	$+2.879 \pm 0.070$	0.180 ± 0.029
20	3.106 ± 0.073	0.227 ± 0.019	$+2.879 \pm 0.070$	0.112 ± 0.009

Table 26: Transport closes 93% of the distance to the true target coordinate. At one target sample the transported coordinate is *closer to the truth* than the coordinate encoded from that sample, and abundant evidence overtakes it between five and twenty—exactly the claimed behaviour. The first three columns cannot vary with K_T , since both coordinates derive from source data alone.

12 Reproduction procedure

The steps below reproduce the image experiments exactly. Every random choice derives from one seed, so a fresh machine following these steps obtains the same splits, the same episodes and the same numbers.

Step 1 — environment and data

Python 3.11 or later, PyTorch with CUDA, about 2 GB of GPU memory. Place the official `cifar-100-python` directory where `domains/cifar100.py` expects it. Verify the installation with the five test modules—72 assertions, including a conformance suite that checks the implementation against the written specification clause by clause.

```
python -m tests.test_score_identity      python -m tests.test_backbone_swap
python -m tests.test_gmm_analytic        python -m tests.test_cifar100_episodes
python -m tests.test_document_conformance
```

Step 2 — build the split

All 60,000 images are pooled into one array indexed $0 \dots 59,999$; the official train/test boundary is discarded, because what is being divided is semantic classes. Then, from `seed = 12345`:

1. Permute the twenty superclass identifiers with a generator keyed by `(seed, 0)`. Take the first twelve as training, the next four as validation, the last four as test. Every fine class inherits its superclass’s split.
2. For each fine class independently, permute its 600 image identifiers with a generator keyed by `(seed, 2000 + fine_id)`. Keying per class means changing one configuration value never disturbs another class’s pools.
3. Cut that permutation at 350: the first 350 identifiers are the source pool and stay clean; the remaining 250 are the target pool and will be transformed.
4. Cut the source pool at 300 into source support and source query. Cut the target pool at 20 into a target support reserve and target query.

Assert that the four index sets are pairwise disjoint and that every image identifier in a class carries that class’s label. Both checks live in `episodes/guards.py` and run by default.

```
python -m runner.build_cifar100_splits
```

Step 3 — draw one episode

An episode is one adaptation rehearsal, determined by the triple (fine class, transformation, target-support size).

1. Sample a fine class uniformly from the current split’s classes.
2. Sample the transformation from the configured set—a single fixed one in the main experiment, so this step is trivial there.
3. Sample K_T from $\{1, 2, 5, 10, 20\}$, or fix it when sweeping.
4. Draw M_S identifiers from the source support pool without replacement, and 32 from the source query pool. Fetch their pixels and scale to $[-1, 1]$. **Apply no transformation.**
5. Take the *first* K_T identifiers of the target support reserve—first, not sampled, so support sets are nested and the K_T sweep is a within-group comparison. Draw 32 identifiers from the target query pool. Fetch pixels, scale to $[-1, 1]$, then **apply the transformation**, seeding any randomness from the image identifier.

At the default settings one episode touches 133 of the class’s 600 images. The pools never change; only which members are drawn changes from one episode to the next, so one class yields an effectively unlimited number of distinct episodes.

Step 4 — one training step

1. Encode the source support to z_S and the target support to z_T^{enc} , through the *same* encoder. Neither support set enters the diffusion forward process—support images only produce coordinates. They do carry the image transformation if they belong to the target domain; the two are separate operations.
2. Compute $\tilde{z}_T = z_S + \Delta_\gamma(z_S, c)$. Pass z_S and the relation descriptor only; no target tensor enters this call.
3. Apply the diffusion forward process to the source query batch once, then evaluate L_{src} with z_S .
4. Apply the forward process to the target query batch *once*, then evaluate both L_{tgt} (with z_T^{enc}) and L_{trans} (with $\tilde{z}_T$) on that one forward-process draw, through a single backbone pass. Sharing the draw makes their difference a paired estimate; sharing the pass halves the cost.
5. Form L_{meta} , back-propagate, clip the gradient norm at 1.0, step AdamW at 2×10^{-4} with 500 warmup steps, and update the exponential moving average of all parameters at decay 0.999.

```
python -m runner.train --scheme domainshift --steps 50000
```

Step 5 — diagnostics during training

Every 1000 steps, on **validation** episodes, under the moving-average weights, evaluate six coordinates on one shared forward-process draw: the episode’s own, zero, another episode’s, a random vector of matched norm, the target-encoded one, and the mean across episodes. Report:

- r_{basis} —the magnitude of the basis contribution relative to the baseline prediction. Near zero means the basis is unused.
- *gain vs $z = 0$ —necessary but not sufficient.* It answers whether the model needs a non-zero coordinate, not whether it uses the coordinate to identify the task.

- $\text{task_specific_frac} = \frac{\text{loss}[\text{mean } z] - \text{loss}[\text{own } z]}{\text{loss}[z=0] - \text{loss}[\text{own } z]}$ —**the decisive quantity**. Near zero means the basis is a constant offset regardless of how healthy the gain looks.
- coordinate spread across episodes, source-to-target coordinate separation, and transport displacement.

Read the third of these before the second. Taking *gain vs $z = 0$* at face value produced two wrong conclusions in this programme, each of which survived several days.

Step 6 — deployment-time evaluation

Freeze ϕ, ψ, γ . For each held-out episode, run every strategy on the *same* episode with the *same* noise draw, so differences are paired:

1. **no adaptation**— $z = 0$, no refinement.
2. **target only**—initialise at z_T^{enc} , prior centre itself. The source-free fallback.
3. **reuse the source coordinate**—initialise at z_S .
4. **transport without refinement**— $\tilde{z}_T$, zero refinement steps. This one is target-free: it touches no target image at all.
5. **transport with refinement**—the method: initialise and centre the prior at $\tilde{z}_T$, then refine by equation (27) on the K_T target support images.
6. **oracle**—refine on abundant target data. Not available at deployment; an attainable upper bound only.

All strategies share one budget object, so the matched comparison holds structurally rather than by anyone remembering it. Report the mean over at least 32 episodes with 95% confidence intervals on the *per-episode difference*.

```
python scripts/cifar_table.py checkpoints/<run>_step50000.pt --split val
```

Step 7 — protocol constraints to preserve

- Choose every hyperparameter and adaptation budget on **validation classes only**, then freeze. Selecting on test makes the reported number the maximum over choices rather than an estimate of held-out performance.
- Target query data must never enter refinement. This is enforced at the type level: the refinement function has no parameter through which it could arrive.
- The relation descriptor is indexed by *relation*, never by semantic class or any grouping disjoint between splits—otherwise meta-testing reads embedding vectors that were never trained, and the transport map is silently disabled.
- Read any “this method attains X ” claim only where the curve against training steps has levelled. The same quantity differed by more than twenty percentage points between an undertrained and a converged run in this programme.

Step 8 — three implementation traps

- A standard deviation computed as $\sqrt{\text{var}}$ rather than $\sqrt{\text{var} + \epsilon}$: a single-element support set has exactly zero variance, and the gradient of $\sqrt{0}$ is infinite. The forward pass is finite, which is what makes this expensive to find—the model becomes NaN within a few steps.
- A basis head initialised at exactly zero pins $\partial L / \partial z$ at nought, so the encoder receives no gradient through the basis path and never begins learning. Initialise small but non-zero.
- Evaluation scripts that rebuild the model from current defaults rather than the checkpoint’s own configuration. This fails loudly when dimensions differ and *silently* when they happen to match.